6. Multimodal Video Analysis Application

6. Multimodal Video Analysis Application

- 1. Concept Introduction
  - 1.1 What is "Video Analysis"?
  - 1.2 Brief Description of the Principle
- 2. Code Analysis
  - Key Code
    - 1. Tool Layer Entry (`largemodel/utils/tools_manager.py`)
    - 2. Model Interface Layer and Frame Extraction (`largemodel/utils/large_model_interface.py`)
  - Code Analysis
- 3. Practical Operation
  - 3.1 Configure Offline Large Model
    - 3.1.1 Configure LLM Platform (`yahboom.yaml`)
    - 3.1.2 Configure Model Interface (`large_model_interface.yaml`)
  - 3.2 Start and Test the Function

Note: The RDK X5 4GB version cannot run this offline due to performance limitations. Please refer to the tutorial for online large models.

1. Concept Introduction

1.1 What is "Video Analysis"?

In the `largemodel` project, the **multimodal video analysis** function refers to the robot's ability to process a video and summarize its core content, describe key events, or answer specific questions about the video in natural language. This elevates the robot from understanding only static images to understanding a dynamic, temporally related world.

The core tool for this function is `analyze_video`. When the user provides a video file and asks a question (e.g., "summarize what this video is about"), the system calls this tool to process and analyze the video, and returns the AI's text-based answer.

1.2 Brief Description of the Principle

The core challenge of offline video analysis is how to enable the model to efficiently process video data containing hundreds or thousands of frames. A mainstream implementation principle is as follows:

1. **Keyframe Extraction**: First, the system does not process every frame of the video. Instead, it extracts the most representative **keyframes** through algorithms (such as scene boundary detection or fixed time interval sampling). This greatly reduces the amount of data that needs to be processed.
2. **Image Encoding**: Each extracted keyframe, like in a visual understanding application, is fed into a visual encoder and converted into a numerical vector containing image information.
3. **Temporal Information Fusion**: This is the biggest difference from single-image understanding. The model needs to understand the time sequence of these keyframes. Usually, a recurrent neural network (RNN) or Transformer model is used to fuse the vectors of all keyframes to form a "memory" vector that can represent the dynamic content of the

entire video.

4. **Question Answering and Generation**: After the user's text question is encoded, it is cross-modally fused with this "memory" vector. Finally, the language model part generates a summary of the entire video or an answer to a specific question based on this fused information.

In simple terms, it's about **"condensing" a video into a few key pictures and their sequence, then understanding the whole story like looking at a comic strip, and answering related questions**.

---

2. Code Analysis

Key Code

1. Tool Layer Entry (`largemodel/utils/tools_manager.py`)

The `analyze_video` function in this file defines the execution flow of the tool.

```
# From largemodel/utils/tools_manager.py
class ToolsManager:
    # ...
    def analyze_video(self, args):
        """
        Analyze video file and provide content description.

        :param args: Arguments containing video path.
        :return: Dictionary with video description and path.
        """
        self.node.get_logger().info(f"Executing analyze_video() tool with args:
{args}")
        try:
            video_path = args.get("video_path")
            # ... (Intelligent path fallback mechanism)

            if video_path and os.path.exists(video_path):
                # ... (Build Prompt)

                # Use a fully isolated, one-time context for video analysis to
ensure a plain text description.
                simple_context = [{
                    "role": "system",
                    "content": "You are a video description assistant. ..."
                }]

                result = self.node.model_client.infer_with_video(video_path,
prompt, message=simple_context)

                # ... (Process result)
                return {
                    "description": description,
                    "video_path": video_path
                }
            # ... (Error handling)
```

2. Model Interface Layer and Frame Extraction (`largemodel/utils/large_model_interface.py`)

The functions in this file are responsible for processing the video file and passing it to the underlying model.

```
# From largemodel/utils/large_model_interface.py

class model_interface:
    # ...
    def infer_with_video(self, video_path, text=None, message=None):
        """Unified video inference interface."""
        # ... (Prepare message)
        try:
            # Decide which specific implementation to call based on
self.llm_platform
            if self.llm_platform == 'ollama':
                response_content = self.ollama_infer(self.messages,
video_path=video_path)
            # ... (Logic for other online platforms)
        # ...
        return {'response': response_content, 'messages': self.messages.copy()}

    def _extract_video_frames(self, video_path, max_frames=5):
        """Extract keyframes from a video for analysis."""
        try:
            import cv2
            # ... (Video reading and frame interval calculation)
            while extracted_count < max_frames:
                # ... (Loop to read video frames)
                if frame_count % frame_interval == 0:
                    # ... (Save frame as a temporary image)
                    frame_base64 = self.encode_file_to_base64(temp_path)
                    frame_images.append(frame_base64)
            # ...
            return frame_images
        # ... (Exception handling)
```

Code Analysis

The implementation of the video analysis function is one step more complex than image analysis. It requires a key preprocessing step at the model interface layer: frame extraction.

1. **Tool Layer (`tools_manager.py`)**:
    - The `analyze_video` function is the business entry point for video analysis. Its responsibility is clear: to receive a video file path and construct a prompt for requesting a description of the video content.
    - It starts the analysis process by calling the `self.node.model_client.infer_with_video` method. Like the visual understanding tool, it is completely unconcerned with the underlying model details and is only responsible for passing the "video file" and "analysis instruction."
2. **Model Interface Layer (`large_model_interface.py`)**:

- The `infer_with_video` function is the scheduler that connects the upper-level tools and the underlying model. It dispatches tasks to the corresponding specific implementation functions based on the platform configuration (`self.llm_platform`).
- Unlike processing a single image, processing a video requires an additional step. The `_extract_video_frames` method demonstrates the general logic for implementing this step: it uses the `cv2` library to read the video and extract a number of keyframes (default is 5).
- Each extracted frame is treated as an independent image and is usually encoded as a Base64 string.
- Finally, the request containing multiple frame image data and analysis instructions is sent to the large model. The model will comprehensively analyze these continuous images to generate a description of the entire video content.
- This "video-to-multiple-images" preprocessing is completely encapsulated in the model interface layer and is transparent to the tool layer.

In summary, the general flow of video analysis is: `ToolsManager` initiates an analysis request -> `model_interface` intercepts the request and calls `_extract_video_frames` to decompose the video file into multiple keyframe images -> `model_interface` sends these images along with the analysis instructions to the corresponding model platform according to the configuration -> the model returns a comprehensive description of the video -> the result is finally returned to `ToolsManager`. This design ensures the stability and versatility of the upper-level application.

3. Practical Operation

3.1 Configure Offline Large Model

3.1.1 Configure LLM Platform (`yahboom.yaml`)

This file determines which large model platform the `model_service` node loads as its primary language model.

1. **Open the file in the terminal**:

```
vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

2. **Modify/Confirm** `llm_platform`:

```
model_service:                          # Model server node parameters
  ros__parameters:
    language: 'en'                      # Large model interface language
    useolinetts: True                   # This item is invalid in text mode
and can be ignored

    # Large model configuration
    llm_platform: 'ollama'              # Key: Make sure this is 'ollama'
    regional_setting : "China"
```

3.1.2 Configure Model Interface (`large_model_interface.yaml`)

This file defines which vision model to use when the platform is selected as `ollama`.

1. Open the file in the terminal

```
vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

2. Find the ollama-related configuration

```
#.....
## Offline Large Language Models
# Ollama configuration
ollama_host: "http://localhost:11434"  # Ollama server address
ollama_model: "llava"            # Key: Change this to your downloaded multimodal
model, e.g., "llava"
#.....
```

Note: Please ensure that the model specified in the configuration parameters (e.g., `llava`) can handle multimodal input.

3.2 Start and Test the Function

Note: Using a model with a small number of parameters or running with limited memory will result in poor performance. For a better experience, please refer to the corresponding chapter in <Online Large Model (Text Interaction)>.

1. **Prepare the video file**:

   Place a video file to be tested in the following path:
   `~/yahboom_ws/src/largemodel/resources_file/analyze_video`

   Then name the video `test_video.mp4`

2. **Start the `largemodel` main program**:

   Open a terminal and run the following command:

   ```
   ros2 launch largemodel largemodel_control.launch.py
   ```

3. **Test**:
   - **Wake up**: Say to the microphone: "Hi,Yahboom."
   - **Conversation**: After the speaker responds, you can say: `analyze the video content`
   - **Observe the log**: In the terminal where the `launch` file is running, you should see:
     1. The ASR node recognizes your question and prints it out.
     2. The `model_service` node receives the text, calls the LLM, and prints the LLM's reply.
   - **Listen to the answer**: After a while, you should be able to hear the answer from the speaker.